Name: P ARCHITH Roll Number: 2303A52258 Batch: 45

Assignment Number:19.3(Present assignment number)/**24**(Total number of assignments)

Lab 19 – Code Translation: Converting Between Programming Languages

Lab Objectives:

- Understand how AI tools can assist in translating code between different programming languages.
- Learn to verify correctness and functionality after translation.
- Explore syntactic and semantic differences between languages (e.g., Python, Java, C++).
- Practice debugging and optimizing AI-translated code.

Task 1 – Python to C++ Conversion

Task:

Translate a simple Python class into C++ using AI assistance.

Instructions:

- Prompt AI to convert a Python class representing a Student with attributes and methods into equivalent C++ code
- Ensure proper handling of:
  - Constructors
  - Data types
  - Access specifiers
- Compile and run both versions to verify output consistency

Expected Output:

An equivalent working C++ class translated from Python.

Prompt:

Generate a python code for adding two numbers
now convert this python code into c++

Code:

```python
def add_two_numbers(a, b):
    """Add two numbers and return the result"""
    return a + b
# Example usage
num1 = 10
num2 = 20
result = add_two_numbers(num1, num2)
print(f"The sum of {num1} and {num2} is {result}")
```

```cpp
#include <iostream>
using namespace std;

int add_two_numbers(int a, int b) {
    // Add two numbers and return the result
    return a + b;
}
int main() {
    int num1 = 10;
    int num2 = 20;
    int result = add_two_numbers(num1, num2);
    cout << "The sum of " << num1 << " and " << num2 << " is " << result << endl;
    return 0;
}
```

Output:

```
PS D:\3-2\AI Assitant Coding\Code Files>  & 'C:\Users\SNEHITH\AppData\Local\Programs\
ons\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '65402' '--'
The sum of 10 and 20 is 30
PS D:\3-2\AI Assitant Coding\Code Files>
```

Task 2 – Java to Python Function Conversion

Task:

Convert a Java method that checks whether a number is prime into Python.

Instructions:

- Ask AI to translate a Java isPrime() method into Python
- Test the function with multiple inputs
- Ensure the Python version follows Pythonic syntax and logic

Expected Output:

A correct Python function equivalent to the original Java prime-checking method.

Prompt:

Generate a python code to check the given number is prime or not

Convert this python code into java

Code:

```python
def is_prime(n):
    """Check whether a number is prime"""
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False
    for i in range(3, int(n**0.5) + 1, 2):
        if n % i == 0:
            return False
    return True

# Test the function
test_numbers = [2, 3, 4, 10, 17, 20, 29]
for num in test_numbers:
    print(f"{num} is prime: {is_prime(num)}")
```

```java
public class PrimeChecker {
        public static boolean isPrime(int n) {
            if (n < 2) {
                return false;
            }
            if (n == 2) {
                return true;
            }
            if (n % 2 == 0) {
                return false;
            }
            for (int i = 3; i <= Math.sqrt(n); i += 2) {
                if (n % i == 0) {
                    return false;
                }
            }
            return true;
        }
        public static void main(String[] args) {
            int[] testNumbers = {2, 3, 4, 10, 17, 20, 29};
            for (int num : testNumbers) {
                System.out.println(num + " is prime: " + isPrime(num));
            }
        }
    }
```

Output:

```
PS D:\3-2\AI Assitant Coding\Code Files>  d:; cd 'd:\3-2\AI Assitant Coding\Code Files
3\python.exe' 'c:\Users\SNEHITH\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x
itant Coding\Code Files\TAsk-19.3.py'
2 is prime: True
3 is prime: True
4 is prime: False
10 is prime: False
17 is prime: True
20 is prime: False
29 is prime: True
PS D:\3-2\AI Assitant Coding\Code Files>
```

Task 3 – Pseudocode to Python Implementation

Task:

Translate a given pseudocode algorithm into Python using AI.

Instructions:

- Provide AI with pseudocode for sorting numbers (e.g., bubble sort or selection sort)
- Ask AI to convert it into executable Python code
- Validate correctness using sample input lists

Expected Output:

A working Python program that correctly implements the pseudocode logic.

Prompt:

Generate a pseudocode algorithm of bubble sort

Convert this into into a full running python code

Code:

```python
# Pseudocode for Bubble Sort
# ALGORITHM BubbleSort(arr)
# INPUT: arr - list of numbers to sort
# OUTPUT: sorted array in ascending order
#
# FOR i = 0 TO length(arr) - 1 DO
#     FOR j = 0 TO length(arr) - i - 2 DO
#         IF arr[j] > arr[j + 1] THEN
#             SWAP arr[j] with arr[j + 1]
#         END IF
#     END FOR
# END FOR
# RETURN arr
def bubble_sort(arr):
    """Sort an array using bubble sort algorithm"""
    n = len(arr)
    for i in range(n):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr
# Test with sample input
test_list = [64, 34, 25, 12, 22, 11, 90]
print(f"Original list: {test_list}")
sorted_list = bubble_sort(test_list)
print(f"Sorted list: {sorted_list}")
```

Output:

```
PS D:\3-2\AI Assitant Coding\Code Files>  d:; cd 'd:\3-2\AI Assitant Coding\C
3\python.exe' 'c:\Users\SNEHITH\.vscode\extensions\ms-python.debugpy-2025.18.
itant Coding\Code Files\TAsk-19.3.py'
Original list: [64, 34, 25, 12, 22, 11, 90]
Sorted list: [11, 12, 22, 25, 34, 64, 90]
PS D:\3-2\AI Assitant Coding\Code Files>
```

Task 4 – SQL to Pandas Query

Task:

Translate a **SQL query** into a **Pandas DataFrame operation** in Python.

Instructions:

- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

Expected Output:

- Equivalent Pandas query that retrieves the correct subset of data.

Prompt:

Generate a sql query returning thr salary >50000

Convert this into a pandas dataframe operation that retrives correct subset of data

Code:

```
# SQL Query: SELECT name, salary FROM employees WHERE salary > 50000

# Sample DataFrame
employees = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'salary': [60000, 45000, 75000, 50000, 55000]
})


# Pandas equivalent - filter rows where salary > 50000 and select name, salary columns
result = employees[employees['salary'] > 50000][['name', 'salary']]
print(result)
```

Output:

```
PS D:\3-2\AI Assitant Coding\Code Files> & C:\Users\SNEHITH\AppData\Local\Programs\P
s/TAsk-19.3.py"
      name  salary
0    Alice   60000
2  Charlie   75000
4      Eve   55000
PS D:\3-2\AI Assitant Coding\Code Files>
```

Task 5 – Real-Time Application: Algorithm Translation Across Languages
Scenario:
A company maintains algorithms written in different programming languages.
Examples:

- Converting a Python searching algorithm into C
- Translating a Java sorting program into JavaScript

Instructions:

- Use AI to translate a selected algorithm between two programming languages
- Execute and test both versions
- Document translation challenges such as:
  - Syntax differences
  - Library support
  - Memory management

Expected Output:
Equivalent, tested code implementations in two different programming languages.

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

Prompt:

Generate a python code of binary search in python

Generate a python code of binary search in c

Generate a python code of binary search in java

Code:

```python
def binary_search(arr, target):
    """Search for target in sorted array using binary search"""
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
# Test with sample input
test_array = [2, 5, 8, 12, 16, 23, 38, 45, 56, 67, 78]
target = 23
result = binary_search(test_array, target)
print(f"Array: {test_array}")
print(f"Target: {target}")
print(f"Index: {result}" if result != -1 else f"Element not found")
```

```java
public class BinarySearch {
    public static int binarySearch(int[] arr, int target) {
        // Search for target in sorted array using binary search
        int left = 0, right = arr.length - 1;
        while (left <= right) {
            int mid = (left + right) / 2;
            if (arr[mid] == target) {
                return mid;
            } else if (arr[mid] < target) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return -1;
    }
    public static void main(String[] args) {
        // Test with sample input
        int[] testArray = {2, 5, 8, 12, 16, 23, 38, 45, 56, 67, 78};
        int target = 23;
        int result = binarySearch(testArray, target);

        System.out.print("Array: ");
        for (int num : testArray) {
            System.out.print(num + " ");
        }
        System.out.println();
        System.out.println("Target: " + target);
```

```c
#include <stdio.h>
int binary_search(int arr[], int size, int target) {
    // Search for target in sorted array using binary search
    int left = 0, right = size - 1;
    while (left <= right) {
        int mid = (left + right) / 2;
        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return -1;
}
int main() {
    // Test with sample input
    int test_array[] = {2, 5, 8, 12, 16, 23, 38, 45, 56, 67, 78};
    int size = sizeof(test_array) / sizeof(test_array[0]);
    int target = 23;
    int result = binary_search(test_array, size, target);

    printf("Array: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", test_array[i]);
    }
    printf("\n");
    printf("Target: %d\n", target);
```

Output:

```
PS D:\3-2\AI Assitant Coding\Code Files>  d:; cd 'd:\3-2\AI Assitant Coding\Code
3\python.exe' 'c:\Users\SNEHITH\.vscode\extensions\ms-python.debugpy-2025.18.0-w
itant Coding\Code Files\TAsk-19.3.py'
Array: [2, 5, 8, 12, 16, 23, 38, 45, 56, 67, 78]
Target: 23
Index: 5
PS D:\3-2\AI Assitant Coding\Code Files>
```